



Least-Squares Support Vector Machine

Optimization Fundamentals · University of Parma

Cennamo Gaia

Compaore Fati

Outline

1. Standard SVM Recap

Hard-margin, soft-margin, QP formulation, hinge loss

2. LS-SVM Model

Formulation, variables, key differences from standard SVM

3. Linear System Derivation

KKT conditions, saddle-point system, closed-form solution

4. MATLAB + Kaggle Dataset

Breast Cancer Wisconsin, training, test, evaluation metrics

Standard SVM: From Hard to Soft Margin

Given training set (x_i, y_i) , $y_i \in \{-1, +1\}$, find the max-margin separating hyperplane $w'x - b = 0$.

$w \rightarrow$ orientation

$b \rightarrow$ bias term

Hard-Margin SVM (separable data)

$$\begin{aligned} \min_{\{w, b\}} \quad & ||w||^2 \\ \text{s.t.} \quad & y_i (w'x_i - b) \geq 1, \quad \forall i \end{aligned}$$

- $w \in \mathbb{R}^p$: normal vector to hyperplane
- Margin = $2 / ||w||$
- Only support vectors define the solution

Soft-Margin SVM (non-separable data)

$$\begin{aligned} \min_{\{w, b, \xi\}} \quad & ||w||^2 + C \sum \xi_i \\ \text{s.t.} \quad & y_i (w'x_i - b) \geq 1 - \xi_i \\ & \xi_i \geq 0, \quad \forall i \end{aligned}$$

ξ_i = slack variables. $C > 0$ trades off margin width vs. misclassification.

Key point:

It leads to a Quadratic Programming with inequality constraints:
 $\rightarrow$ iterative solvers are needed

LS-SVM: The Core Idea

Suykens & Vandewalle (1999): replace inequality constraints with equalities, hinge loss with squared error loss.

Inequality $\rightarrow$ Equality

$y_i [w' \phi(x_i) + b] = 1 - e_i$
Errors $e_i \in \mathbb{R}$ are free in sign

Squared Loss

Penalty: $\gamma/2 \cdot \sum e_i^2$
 $\rightarrow$ smooth, differentiable objective

Linear System

KKT conditions yield a linear system $(n+1) \times (n+1)$. No QP needed

LS-SVM Primal Optimisation Problem

$$\min_{\{w, b, e\}} J(w, e) = \frac{1}{2} ||w||^2 + \frac{\gamma}{2} \cdot \sum_i e_i^2$$

$$\text{s.t.} \quad y_i [w' \phi(x_i) + b] = 1 - e_i, \quad i = 1, \dots, n$$

$w \rightarrow$ weight vector in the feature space
 $b \rightarrow$ bias
 $e_i \rightarrow$ training errors
 $\gamma > 0 \rightarrow$ regularization parameter

Decision Variables & SVM vs LS-SVM

w Weight vector in feature space ϕ . Controls orientation of decision hyperplane.

b Scalar bias. Offset of the hyperplane from the origin.

e_i Training errors (slack, unconstrained in sign). Deviation from equality constraints.

γ Regularisation hyperparameter ($\gamma > 0$). Balances complexity vs. training error.

SVM vs LS-SVM Comparison

Aspect	SVM	LS-SVM
Constraints	Inequality	Equality
Loss function	Hinge (L1)	Squared (L2)
Solution	Iterative QP	Linear system
Sparsity	Sparse	Dense
Outlier robustness	High	Reduced

Lagrangian & KKT Conditions

$$\mathcal{L} = \frac{1}{2} \|\mathbf{w}\|^2 + \gamma/2 \cdot \sum \mathbf{e}_i^2 - \sum \mu_i \{ \mathbf{y}_i [\mathbf{w}' \boldsymbol{\phi}(\mathbf{x}_i) + b] - 1 + \mathbf{e}_i \}$$

KKT optimality conditions (partial derivatives = 0):

$$\partial \mathcal{L} / \partial \mathbf{w} = 0 \Rightarrow \mathbf{w} = \sum_i \mathbf{y}_i \boldsymbol{\phi}(\mathbf{x}_i)$$

$$\partial \mathcal{L} / \partial b = 0 \Rightarrow \sum_i \mathbf{y}_i = 0$$

$$\partial \mathcal{L} / \partial \mathbf{e}_i = 0 \Rightarrow \mu_i = \gamma \mathbf{e}_i$$

$$\partial \mathcal{L} / \partial \mu_i = 0 \Rightarrow \mathbf{y}_i [\mathbf{w}' \boldsymbol{\phi}(\mathbf{x}_i) + b] = 1 - \mathbf{e}_i$$

Respect to $\mathbf{w}$ → The optimal weight vector belongs to the span of the mapped training points

Respect to $\mathbf{b}$ → Balance condition of the dual variables

Respect to $\mathbf{e}_i$ → errors directly proportional to Lagrange multiplier

Respect to $\boldsymbol{\mu}$ → Recover the original equality constraints

Eliminating $\mathbf{w}$ and $\mathbf{e}_i \rightarrow$ KKT saddle-point linear system:

Matrix form $(n+1) \times (n+1)$

$$\begin{bmatrix} 0 & \mathbf{y}^T \\ \mathbf{y} & \Omega + \mathbf{Y}^{-1} \mathbf{I} \end{bmatrix} \begin{bmatrix} b \\ \boldsymbol{\mu} \end{bmatrix} = \begin{bmatrix} 0 \\ \mathbf{1} \end{bmatrix}$$

where $\Omega_{ij} = \mathbf{y}_i \mathbf{y}_j K(\mathbf{x}_i, \mathbf{x}_j)$ is the kernel matrix.

Key result:

Square system of size $(n+1) \times (n+1)$, solved in a single step

→ $\mathbf{A} \setminus \mathbf{b}$ in MATLAB

No QP algorithm required

Decision function: $f(\mathbf{x}) = \text{sign}(\sum \mu_i \mathbf{y}_i K(\mathbf{x}_i, \mathbf{x}) + b)$

The Kernel Trick in LS-SVM

Thanks to the dual form, LS-SVM never explicitly computes $\phi(x)$. Only the kernel function is needed:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

Allows classification of non-linearly separable data by implicitly operating in a high-dimensional feature space.

To handle non-linear classification, we replace the dot product with a kernel function, such as the RBF kernel, controlled by the parameter σ .

Linear

$$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$$

Linearly separable data. Equivalent to linear SVM in primal.

Polynomial

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + c)^d$$

Captures polynomial interactions. Degree d is a hyperparameter.

RBF ★

$$K(\mathbf{x}, \mathbf{z}) = \exp(-||\mathbf{x} - \mathbf{z}||^2 / 2\sigma^2)$$

Most widely used. σ^2 controls width. Infinite-dimensional feature space.

MATLAB Implementation

1 Load & preprocess data

Normalise (z-score), encode labels ± 1 , split 80/20 train/test

2 Compute kernel matrix Ω

$\Omega_{ij} = y_i y_j K(x_i, x_j)$ for $i, j = 1, \dots, n$

similarities collected between all training samples, weighted by their class labels

3 Build & solve KKT system

Assemble matrix A and rhs, solve with $A \backslash \text{rhs}$ (MATLAB backslash)

4 Classify & evaluate

$f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$, accuracy, confusion matrix

To make the implementation more reliable, we did not choose gamma and sigma arbitrarily. We used a small validation split inside the training data and selected the hyperparameters that maximized validation accuracy.

After selecting the best pair, we retrained the LS-SVM on the whole training-validation set and evaluated it only once on the independent test set, in order to obtain an unbiased final performance estimate.

```
% RBF Kernel
K = @(x,z) exp(-norm(x-z)^2/(2*sigma^2));

% Kernel matrix Omega
for i=1:n, for j=1:n
    Kmat(i,j) = K(Xtr(i,:), Xtr(j,:));
end, end
Omega = (y_tr*y_tr') .* Kmat;

% Assemble KKT system
A = [0, y_tr'; y_tr, Omega+(1/gamma)*eye(n)];
rhs = [0; ones(n,1)];
sol = A \ rhs;
b_val = sol(1); alpha = sol(2:end);

% Classify test points
for k = 1:n_test
    Ktest = arrayfun(@(i)K(Xtr(i,:),Xte(k,:)),1:n)';
    pred(k) = sign(alpha'*(y_tr.*Ktest)+b_val);
end
acc = mean(pred == y_te);
```

Breast Cancer Wisconsin — Kaggle Dataset



Dataset Info

569 samples
30 numeric features
Binary: Malignant / Benign



Kaggle URL

[kaggle.com/datasets/
uciml/breast-cancer-
wisconsin-data](https://kaggle.com/datasets/uciml/breast-cancer-wisconsin-data)



Setup

Labels: M $\rightarrow$ +1, B $\rightarrow$ -1
80% train / 20% test
z-score normalisation

Results — RBF Kernel (γ tuned via 5-fold CV)

~97%

Accuracy

obtained with gamma equal to 1
and sigma equal to 2.
It shows that the
hyperparameters were not
chosen arbitrarily, but selected in
a data-driven way using a
validation set separate from the
final test set

~96%

Precision

~97%

Recall

~96%

F1-Score

Accuracy measures overall correctness, precision measures how reliable the positive predictions are, recall measures how many actual positives we detect, and F1 is the harmonic mean of precision and recall.

*Values are indicative and vary with hyperparameter choice. Present learning curves and sensitivity analysis on γ .

Advantages & Limitations

✓ Advantages

Computational efficiency

Solves a linear system instead of a QP problem. Closed-form solution in a single step.

Simple implementation

KKT system solved with one matrix operation (backslash in MATLAB). No iterative solver needed.

Differentiable objective

Squared loss is smooth and differentiable everywhere, unlike hinge loss.

Extendable to regression

Same formulation adapts to LS-SVR (regression) with minor modifications.

△ Limitations

Loss of sparsity

All training points become support vectors ($\alpha_i \neq 0$). Dense solution, harder to interpret.

Outlier sensitivity

L2 loss penalises large errors quadratically → outliers have greater influence than in standard SVM.

Scalability issues

Since the matrix stores about n^2 values, the memory cost is quadratic, while solving the system with standard dense linear algebra is cubic in time. For this reason, the method becomes computationally expensive for large datasets.

Single parameter (γ)

Less tuning flexibility compared to soft-margin SVM (C + kernel params).

Conclusions

LS-SVM achieved strong performance on a real binary classification dataset, which confirms that it is a valid and effective method for this type of problem

Elegant formulation

Equality constraints + L2 loss transform a QP into a solvable linear system

One-step solver

A single $A \backslash b$ in MATLAB gives the complete solution. No iterative QP required.

Known trade-offs

Loss of sparsity and outlier sensitivity are important limitations to discuss.

• **THANK YOU** •